SUMASIFY Backend — PRD: Uncompleted Modules (v1)

Field	Value
Product	SUMASIFY Marketplace Backend
Document type	Product Requirements Document (gap closure)
Baseline	`API_CONTRACT.md` v1 (in-repo) and `SUMASIFY Backend API Contract.pdf` v1
Code baseline reviewed	branch `origin/feature/buyerorders` (`9164b4d`)
Scope	All contract requirements **not yet implemented**
Status of this doc	Draft for implementation planning

---

1. Summary

The current backend implements authentication, categories, public catalogue, cart, read-only buyer orders, bank-transfer payment submission, and the health check. That is approximately **21 of 48 contract endpoints (~44%)**.

This PRD defines the remaining work required to make the marketplace functional end-to-end:

1. **Checkout and order creation** (highest priority — the buyer flow is currently blocked).
2. **Vendor lifecycle** — applications, profile, products, inventory, image uploads, order fulfilment.
3. **Admin operations** — dashboard, vendor approvals, product moderation, user management, order oversight, payment verification, settlement.
4. **Notifications.**
5. **Cross-cutting hardening** — rate limiting, casing consistency, audit logging, request IDs.

The Prisma schema already models nearly all of these domains (`VendorApplication` , `Vendor` , `Product` , `VendorOrderGroup` , `OrderItem` , `StockReservation` , `Payment` , `AdminAuditLog` , `Notification` , etc.). Most of the work is service/API logic, not data modelling.

---

2. Goals and non-goals

Goals

- Enable a buyer to complete a full purchase: cart → checkout preview → order creation → payment submission → admin verification → fulfilment → settlement.
- Enable a buyer to apply to become a vendor and, once approved, manage products, inventory and fulfilment.

- Enable admins to operate the marketplace: approve vendors, moderate products, verify payments, settle vendors.
- Deliver in-app notifications for order/payment/vendor events.
- Enforce server-authoritative pricing, stock and settlement amounts.
- Meet contract security requirements (rate limiting, ownership checks, audit logs).

Non-goals (explicitly out of V1 scope)

- Chat, reviews, recommendations, delivery routing/maps.
- Paystack/webhooks or automated payouts.
- Platform-managed logistics.
- Automated refunds, fraud scoring, loyalty, promoted listings, multi-country.
- Dedicated `Settlement` model or settlement history/batching.

---

3. Roles and authorization

Role	Description
`buyer`	Default registered user. Can shop, checkout, pay, apply to be a vendor.
`vendor`	Buyer whose application was approved. Manages own products and fulfilment.
`admin`	Platform operator. `adminRole` is `ADMIN` or `SUPER_ADMIN`.

Rules:

- Ownership is always enforced server-side. A vendor may only access its own products/order groups; a buyer may only access its own orders/notifications.
- Admin-only routes require `authenticate` + `authorizeRoles("admin")`; settlement additionally requires `adminRole` in (`ADMIN`, `SUPER_ADMIN`).
- Fulfilment starts only after SUMASIFY verifies payment. Vendors cannot verify payments, modify payment records, or settle themselves.
- Existing middleware to reuse: `authenticate`, `authorizeRoles`, `AppError`, `asyncHandler`, `errorHandler`.

Note: the schema has `UserRole.ADMIN` but no separate vendor role assumption beyond `VENDOR`; vendor access should additionally verify the `Vendor.status === "ACTIVE"`.

---

4. Convention decision (must resolve before coding)

There is a conflict between the two contract artifacts:

- In-repo `API_CONTRACT.md §1.1` mandates **lowercase** API identifiers (`fullname`, `pagesize`, `ordernumber`, `orderid`).
- The attached PDF uses **camelCase** (`fullName`, `pageSize`, `orderNumber`, `orderId`).

The existing code is **inconsistent** (e.g. `fullname` / `accountstatus` / `pagesize` lowercase, but `accessToken` / `expiresIn` / `requestId` camelCase).

Decision required: pick one convention and apply it globally to all new endpoints and retrofit existing ones. For this PRD, examples use the in-repo lowercase convention. If the team standardizes on camelCase instead, every field below must be adapted consistently. Do not ship a mix.

Error envelope (fixed)

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "fields": [{ "field": "email", "message": "Invalid email" }],
    "requestid": "req_123"
  }
}
```

Success envelope (fixed)

```
{ "data": {}, "meta": { "page": 1, "pagesize": 20, "total": 0, "totalpages": 0 } }
```

Shared rules

- Money is returned as decimal strings (`"20000.00"`), never floats.
- `pagesize` max 50, default 20.
- All state-changing checkout/payment requests require `Idempotency-Key`.
- All privileged actions create an `AdminAuditLog` row.

---

5. Current status snapshot

Module	Contract endpoints	Implemented	Status
Auth	8	8	Done
Categories	4	4	Done
Catalogue	2	2	Done
Cart	3	3	Done
Orders (read)	2	2	Done
Payment submission	1	1	Done
Health	1	1	Done (shape needs fix)
Checkout	2	0	**Missing**

Module	Contract endpoints	Implemented	Status
Vendor applications	2	0	**Missing**
Vendor profile	2	0	**Missing**
Vendor products	3	0	**Missing**
Vendor inventory	1	0	**Missing**
Product images	1	0	**Missing**
Vendor orders	3	0	**Missing**
Admin dashboard	1	0	**Missing**
Admin vendor applications	2	0	**Missing**
Admin products	3	0	**Missing**
Admin users	3	0	**Missing**
Admin orders	1	0	**Missing**
Admin payment verification	1	0	**Missing**
Admin settlement	1	0	**Missing**
Notifications	2	0	**Missing**
Rate limiting	cross-cutting	0	**Missing**

---

6. Functional requirements by epic

Each epic follows the existing module pattern: `src/modules/<area>/<area>.{routes,controller,service,model,validation}.js`, registered in `src/routes/index.js`, tested under `tests/*.test.js` with `node --test`.

EPIC 1 — Checkout and Order Creation (P0, critical path)

Why: Orders cannot currently be created, so the buyer journey and payment submission cannot be exercised in production. This unblocks vendors, admin payment verification, and settlement.

1.1 `POST /checkout/preview` (Buyer)

- Input: `addressid`, `deliverypreferences` (`{ vendorId: { method: "DELIVERY" | "PICKUP" } }`).
- Load the buyer's cart; validate address ownership; validate products are active/available; recheck stock.
- Recalculate all prices from the database; never trust client totals.
- Calculate delivery per vendor group; calculate commission server-side; compute payout.
- Response: `currency`, `subtotalamount`, `deliveryamount`, `totalamount`, `vendorgroups[]` (vendor, items, per-group subtotal/delivery/method), `warnings[]`.
- Errors: `400` invalid address/delivery preference; `409` stock changed; `409` product unavailable.

Acceptance criteria

- A buyer with a valid cart and address receives server-calculated totals matching DB prices.
- Modifying a product price between cart add and preview changes the preview total.
- Out-of-stock or inactive products produce `409` with a machine-readable code.
- The buyer cannot preview using another user's address.

1.2 `POST /checkout/orders` (Buyer)

- Required header `Idempotency-Key`.
- Input: `addressid`, `deliverypreferences`, `paymentmethod` (only `BANK_TRANSFER`).
- Execute as **one database transaction**: recheck stock → create parent `Order` → create `VendorOrderGroup`s → create `OrderItem`s (with snapshots) → create `Payment` (`AWAITING_SUBMISSION`) → create `StockReservation`s → commit.
- Server determines: `vendorid`, prices, `commissionamount`, `payoutamount`, subtotal, totals, stock, order number (`SUM-YYYYMMDD-####`).
- On success: `201` with `order`, `payment`, and `paymentinstructions` (platform bank account + `reference = ordernumber`). Do **not** expose vendor bank accounts.
- Clear the buyer's cart on success.
- Idempotency: same key returns the original result; same key with different payload returns `409`.
- Errors: `400` invalid payment method/delivery config; `409` stock conflict/idempotency conflict/unavailable product.

Acceptance criteria

- Duplicate submissions with the same `Idempotency-Key` create exactly one order.
- Concurrent checkout that would oversell fails cleanly and reserves nothing.
- Every created order has at least one vendor group, matching items, a pending payment, and stock reservations.
- `subtotalamount = Σ line totals`; `totalamount = subtotal + delivery`; `payoutamount = subtotal − commission` per vendor group.
- Cart is emptied only after a successful commit.

Implementation notes

- Add `src/modules/checkout/{checkout.routes,controller,service,validation,model}.js` (or place order-creation logic in `orders`).
- Use `prisma.$transaction` with conditional stock updates (`updateMany` with `stockOnHand >= qty`) or row locking to prevent oversell.
- Reuse `formatMoney` helpers for decimal strings.
- Financial snapshot must be stored on `VendorOrderGroup` so later price changes do not alter settlement.

Open dependency — commission rule: the schema stores `commissionAmount` / `payoutAmount` but no commission-rule model/config exists. Define the V1 commission rate (e.g. flat % per category or global) before implementing. Do not invent it silently.

Open dependency — addresses: checkout references `addressid`, but no address CRUD endpoint exists in the contract. Confirm how addresses are created (out-of-scope endpoint, seed, or add `POST/GET /addresses`).

EPIC 2 — Vendor Applications (P0)

2.1 `POST /vendor-applications` (Buyer)

- Input: `businessname`, `contactphone`, `description`, `categorynotes`.
- Reject if the buyer already has a `PENDING` application (`409`).
- Persist `VendorApplication` with `status = PENDING`.
- Response `201` with application DTO.
- Do not require identity-document upload.

2.2 `GET /vendor-applications/me` (Buyer)

- Return the authenticated buyer's latest application, including `status` and `rejectionreason`.

Acceptance criteria

- A buyer cannot submit a second pending application.
- An approved buyer's `User.role` remains `BUYER` until admin approval promotes them (see Epic 8).
- `GET .../me` returns `null` or 404-equivalent when no application exists (confirm contract behavior).

EPIC 3 — Vendor Profile (P0)

3.1 `GET /vendor/profile` (Approved Vendor)

3.2 `PATCH /vendor/profile` (Approved Vendor)

- Editable: `businessname`, `contactphone`, `pickupaddress`, `deliverynotes`.
- Non-editable: vendor id, approval state, owner (`userId`), `status`.
- Reject non-vendor/inactive vendors (`403`).

Acceptance criteria

- A buyer without an active vendor record receives `403`.
- A vendor cannot change ownership or approval fields.

EPIC 4 — Vendor Products and Inventory (P0)

4.1 `GET /vendor/products` (Approved Vendor)

- Paginated, filter by `status`; return only the authenticated vendor's products.

4.2 `POST /vendor/products` (Approved Vendor)

- Input: `name`, `description`, `priceamount`, `categoryid`, `stockonhand`, `imageids[]`.
- Server sets: `vendorId`, `status`, `isAvailable`, `publishedAt`, stock validation.
- Validate category exists and is active; validate upload IDs belong to the vendor and are unused.
- Errors: `400` validation; `403` vendor not approved/active; `404` category/upload not found; `409` image/product constraint conflict.

4.3 `PATCH /vendor/products/:productid` (Owner)

- Editable: `name`, `description`, `priceamount`, `categoryid` (and status toggles if contract allows).

- Reject if product belongs to another vendor (`403`).

4.4 `PATCH /vendor/products/:productid/inventory` (Owner)

- Input: `stockonhand`.
- Enforce `stockOnHand >= 0`, `stockReserved >= 0`, `stockReserved <= stockOnHand`.
- Return `productid`, `stockonhand`, `stockreserved`, `availablestock`.
- Errors: `403` not owner; `404` not found; `409` requested stock below reserved stock.

Acceptance criteria

- Vendors can never read or mutate another vendor's products.
- Product creation is blocked for non-active vendors.
- Inventory cannot drop below reserved stock.
- Deleting/archiving a product does not orphan active orders.

Implementation notes

- Reuse `slugify`/audit patterns from `categories`.
- Product visibility transitions must set `publishedAt` and `isAvailable` correctly.

EPIC 5 — Product Image Uploads (P1)

5.1 `POST /uploads/product-images` (Approved Vendor)

- `multipart/form-data` via `multer`.
- Accept JPEG/PNG/WebP; max 5 MB each; max 5 images per product.
- Validate MIME type, magic bytes, size, ownership, image count, position.
- Upload binary to Cloudinary (dependency already present); persist metadata in `ProductImage` (or a pending-upload record referenced by `imageids`).
- Response: `uploadid`, `url`, `storagekey`, `contenttype`, `width`, `height`, `bytes`.
- Never store binaries in PostgreSQL; never expose private payment proof.

Acceptance criteria

- Non-image and oversized files are rejected.
- Uploaded images can be attached to a product by `uploadid`.
- A vendor cannot attach another vendor's uploads.

Implementation notes

- Two-step flow: upload returns IDs; product create/update references them. Decide whether `ProductImage` rows are created at upload (with nullable product) or at attach time. The current schema requires `productId` and `position`, so a staging table or schema adjustment may be needed.

EPIC 6 — Vendor Orders and Fulfilment (P0)

6.1 `GET /vendor/orders` (Approved Vendor)

- Paginated; filter by status; returns only the vendor's `VendorOrderGroup`s.

6.2 `GET /vendor/orders/:groupid` (Owner)

- Return group, delivery snapshot needed for fulfilment, items, quantities, fulfilment status, delivery method, vendor notes.
- `403` / `404` if not owned.

6.3 `PATCH /vendor/orders/:groupid/status` (Owner)

- Input: `status`, optional `vendornotes`.
- Allowed transitions only: `PENDING_PAYMENT → NEW → CONFIRMED → READY → OUT_FOR_DELIVERY → DELIVERED → COMPLETED`.
- Reject invalid transitions (`409`).
- Vendor cannot verify payment, modify payments, settle, or access another vendor's group.
- Fulfilment may start only after payment verification (group is `NEW`).

Acceptance criteria

- A vendor can only advance status along the allowed path.
- A vendor cannot move a `PENDING_PAYMENT` group forward before payment verification.
- On reaching `DELIVERED` / `COMPLETED`, the group becomes settlement-eligible (Epic 13).
- Parent order status is derived from payment + group states, not client input.

EPIC 7 — Admin Dashboard (P1)

7.1 `GET /admin/dashboard` (Admin)

- Return operational counts:
  - `vendorapplications.pending`
  - `products.active`, `products.pendingmoderation`
  - `orders.pendingpaymentreview`, `orders.infulfilment`, `orders.completed`
  - `payments.submitted`
- Aggregate queries only; no advanced analytics.

Acceptance criteria

- Counts reflect live DB state.
- Non-admins receive `403`.

EPIC 8 — Admin Vendor Applications (P0)

8.1 `GET /admin/vendor-applications` (Admin)

- Paginated; filter by `status`.

8.2 `PATCH /admin/vendor-applications/:id` (Admin)

- Approve: `{ "decision": "approved" }` → create/activate `Vendor`, promote user to `VENDOR`, set application `APPROVED`, set `reviewedBy` / `reviewedAt`.
- Reject: `{ "decision": "rejected", "rejectionreason": "..." }`.

- Enforce valid state transitions; `409` on invalid state.
- Create `AdminAuditLog` on every decision.
- Perform approval transactionally.

Acceptance criteria

- Approval creates exactly one active vendor and one audit record.
- Re-approving/rejecting a finalized application returns `409`.
- A rejected application records the reason and is visible via `GET /vendor-applications/me`.

EPIC 9 — Admin Products (P1)

9.1 `GET /admin/products` (Admin) — list/filter across vendors.

9.2 `GET /admin/products/:productid` (Admin) — view product.

9.3 `PATCH /admin/products/:productid` (Admin) — moderate/update (status/visibility).

- Moderation actions that affect marketplace visibility must write `AdminAuditLog`.

Acceptance criteria

- Admin can deactivate/archive a product; it disappears from public catalogue.
- Audit records capture before/after state.

EPIC 10 — Admin Users (P0)

10.1 `GET /admin/users` (Admin) — list/filter.

10.2 `GET /admin/users/:userid` (Admin) — view.

10.3 `PATCH /admin/users/:userid` (Admin) — update permitted account state.

- Allowed statuses: `active`, `suspended`, `deactivated`.
- Role changes are **not** allowed via this endpoint.
- Admin cannot deactivate self; enforce minimum-admin safety rule.
- Audit all state changes.

Acceptance criteria

- Suspended/deactivated users are blocked at authentication (already supported).
- Self-deactivation and last-admin deactivation are rejected (`409` / `403`).
- Role cannot be escalated through this endpoint.

EPIC 11 — Admin Orders (P1)

11.1 `GET /admin/orders` (Admin)

- Paginated; filter by `status`, `paymentstatus`; return parent orders, vendor groups, items, payment and fulfilment status.

Acceptance criteria

- Admin sees all orders; buyers/vendors do not gain access through this route.

EPIC 12 — Admin Payment Verification (P0)

12.1 `PATCH /admin/payments/:paymentid` (Admin)

- Approve: `{ "decision": "VERIFIED" }`.
- Reject: `{ "decision": "rejected", "rejectionreason": "..." }`.

Verification effects (transactional):

```
payment.status → VERIFIED
payment.verifiedAt → now, verifiedById → admin
order.status → PAID
order.paymentStatus → VERIFIED
relevant VendorOrderGroups (status PENDING_PAYMENT) → NEW
```

Rejection effects:

```
payment.status → REJECTED
order.status → PAYMENT_REJECTED
reserved stock → released (set StockReservation.releasedAt, decrement Product.stockReserved)
```

- Create `AdminAuditLog` for both.
- Do **not** trigger vendor settlement here.
- Order aggregate status must be computed by the order service.

Acceptance criteria

- Verifying one payment produces exactly one audited state change.
- Rejecting releases reserved stock exactly once.
- Payment verification is idempotent/guarded against double-processing.
- Settlement state is untouched by verification.

EPIC 13 — Admin Vendor Settlement (P1)

13.1 `PATCH /admin/vendor-orders/:groupid/settlement` (Admin / Super Admin)

- Body: `{ "status": "SETTLED" }`.
- Preconditions: group exists; `fulfilmentStatus` is `DELIVERED` or `COMPLETED`; `settlementStatus` not already `SETTLED`.
- Amount: `settledAmount = payoutAmount` (stored at checkout; never recalculated from current prices).
- Set `settlementStatus = SETTLED`, `settledAt = now`, `settledAmount = payoutAmount`.
- Transactional; create `AdminAuditLog` (`action: VENDOR_ORDER_SETTLED`).
- Errors: `404 VENDOR_ORDER_NOT_FOUND`; `409 VENDOR_ORDER_NOT_ELIGIBLE_FOR_SETTLEMENT`; `409 VENDOR_ORDER_ALREADY_SETTLED`; `401` / `403` auth.

- Transition to `ELIGIBLE_AFTER_DELIVERY` happens automatically when the group reaches `DELIVERED` / `COMPLETED`.

Acceptance criteria

- Ineligible or already-settled groups cannot be settled.
- Settlement never changes payment verification state.
- Concurrent settlement attempts result in exactly one settlement and one audit record.

Note: the attached PDF does not include this endpoint; it is present in the in-repo `API_CONTRACT.md`. Confirm it is in scope before building.

EPIC 14 — Notifications (P1)

14.1 `GET /notifications` (Authenticated)

- Paginated list of the caller's notifications (`ORDER` , `PAYMENT` , `DELIVERY` , `VENDOR` , `SYSTEM`), newest first.

14.2 `PATCH /notifications/:id/read` (Owner)

- Mark one notification read; `403` / `404` for others' notifications.

Event hooks (create notifications):

- Order created (buyer + each vendor).
- Payment submitted (buyer + admin ops).
- Payment verified/rejected (buyer, vendor).
- Vendor order status changed (buyer).
- Vendor application approved/rejected (buyer).
- Settlement recorded (vendor).

Acceptance criteria

- Users only ever see their own notifications.
- Marking read is idempotent.
- External email/SMS delivery is out of scope.

EPIC 15 — Cross-cutting hardening (P0/P1)

1. **Rate limiting (P0):** add `express-rate-limit` (in-memory) to at least: `POST /auth/register` , `/auth/login` , `/auth/google` , `/auth/forgot-password` , `/auth/reset-password` , `/orders/:orderid/payments/bank-transfer` , and checkout. Return `429` with the standard error envelope.
2. **Casing consistency (P0):** resolve Section 4 and retrofit all endpoints (existing and new) to one convention.
3. **Health check shape (P1):** return `{ "data": { "status": "ok" } }` ; decide whether it lives at `/health` or `/api/v1/health` and align with the contract.
4. `GET /auth/me` **shape (P1):** align to contract (`{ data: { ...user } }`).
5. **Request IDs (P1):** add middleware to read/generate `x-request-id` and expose it consistently in errors and audit logs.

6. **Audit coverage (P0):** every admin action in Epics 8, 9, 10, 12, 13 writes `AdminAuditLog`.
7. **OpenAPI/docs (P2):** optional endpoint documentation once conventions are fixed.

---

7. Data model readiness and required changes

Already present (no migration needed for most epics):

- `VendorApplication`, `Vendor`, `Product`, `ProductImage`, `Cart`, `CartItem`, `Address`, `Order`, `VendorOrderGroup`, `OrderItem`, `StockReservation`, `Payment`, `AdminAuditLog`, `Notification`, and all relevant enums.

Likely schema work:

- **Upload staging:** `ProductImage.productId` is non-nullable; the two-step upload flow needs either a pending-upload model or making `productId` optional until attach. Decide before Epic 5.
- **Commission rule:** no model/config for commission rate; required for checkout/settlement. Add config or a `CommissionRule` model, or define a constant with sign-off.
- **Order number sequence:** ensure a collision-safe generator for `SUM-YYYYMMDD-####` under concurrency.
- **Idempotency storage for checkout:** `Payment.idempotencyKey` exists; checkout-level idempotency may need its own keyed record or reuse of the payment key with a deterministic order reference.
- **Address management:** confirm how `Address` records are created.

---

8. Non-functional requirements

- **Security:** HTTPS in production; bcrypt cost 12 (already in auth); rotated refresh sessions; HttpOnly/Secure/SameSite cookies; server-side role + ownership checks; Zod validation; parameterized Prisma queries; Helmet + CORS allowlist; rate limiting; safe public errors; redacted logs; private payment evidence; audit logging for privileged actions.
- **Concurrency:** checkout and settlement must be transactional; prevent overselling; prevent duplicate settlement; guard payment verification against double-processing.
- **Financial integrity:** all amounts server-calculated and persisted as snapshots; decimal strings in responses; no trust in client totals.
- **Performance:** list endpoints paginated (max 50); dashboard uses aggregate counts; indexes already defined on key query paths.
- **Reliability:** permission failures return `403`, not `404`, only where resource existence is not sensitive; contract-defined status codes honored.
- **Testing:** unit tests for each service/validation; integration tests for checkout transaction, payment verification effects, settlement eligibility, ownership enforcement, and rate limiting. Target meaningful coverage of money/stock/state-machine logic.

---

9. Milestones and phasing

Phase	Contents	Exit criteria
Phase 0 — Decisions	Casing convention, commission rule, address flow, upload staging, settlement scope confirmation	Signed-off decisions recorded in this doc
Phase 1 — Checkout (critical)	Epic 1 + checkout tests	Buyer can create order + pending payment end-to-end; oversell prevented; idempotency proven
Phase 2 — Payment + Admin verification	Epic 12 + audit	Admin can verify/reject; order/vendor-group states update; stock released on reject
Phase 3 — Vendor onboarding	Epics 2, 3, 8	Buyer applies; admin approves; vendor profile accessible
Phase 4 — Vendor catalogue/fulfilment	Epics 4, 5, 6	Vendor manages products/inventory/images and fulfils orders
Phase 5 — Admin operations	Epics 7, 9, 10, 11	Dashboard, product moderation, user management, order oversight
Phase 6 — Settlement + Notifications	Epics 13, 14	Eligible groups settle with audit; notifications emitted and readable
Phase 7 — Hardening	Epic 15	Rate limiting live; casing consistent; health/me fixed; audit coverage verified

10. Definition of done (per endpoint)

- Route registered under `/api/v1` in `src/routes/index.js`.
- Zod validation for params/query/body.
- Service enforces authorization, ownership, and business rules.
- Responses/errors match the contract envelope and chosen casing convention.
- Money returned as decimal strings.
- Audit log written for privileged actions.
- Unit tests (service + validation) and at least one integration/happy-path test.
- `npm test` (`node --test`) passes.
- No secrets committed; logs redacted.

11. Risks and mitigations

Risk	Impact	Mitigation
Casing mismatch between PDF and repo contract	Client integration bugs	Resolve in Phase 0 and standardize globally

Risk	Impact	Mitigation
Commission rule undefined	Checkout/settlement blocked or wrong amounts	Define and approve before Phase 1
Overselling under concurrency	Financial/trust damage	Conditional stock updates/locking inside one transaction
Double settlement / double verification	Financial loss	Transactional eligibility checks + unique constraints + idempotency
Uploads attached across vendors	Data leak/abuse	Ownership checks on upload IDs
Address creation undefined	Checkout blocked	Clarify flow in Phase 0
Scope creep into out-of-V1 features	Delay	Enforce non-goals list

---

12. Success metrics

- Buyer can complete cart → order → payment submission without manual intervention.
- Checkout produces correct totals with zero oversell in concurrency tests.
- Admin verifies a payment and the buyer/vendor see correct derived states.
- Vendor fulfils an order and the group becomes settlement-eligible; admin settles it exactly once with an audit record.
- All new endpoints pass contract-compliance tests and `npm test`.
- Endpoint completion moves from ~44% to 100% of the V1 catalogue (excluding explicitly out-of-scope items).